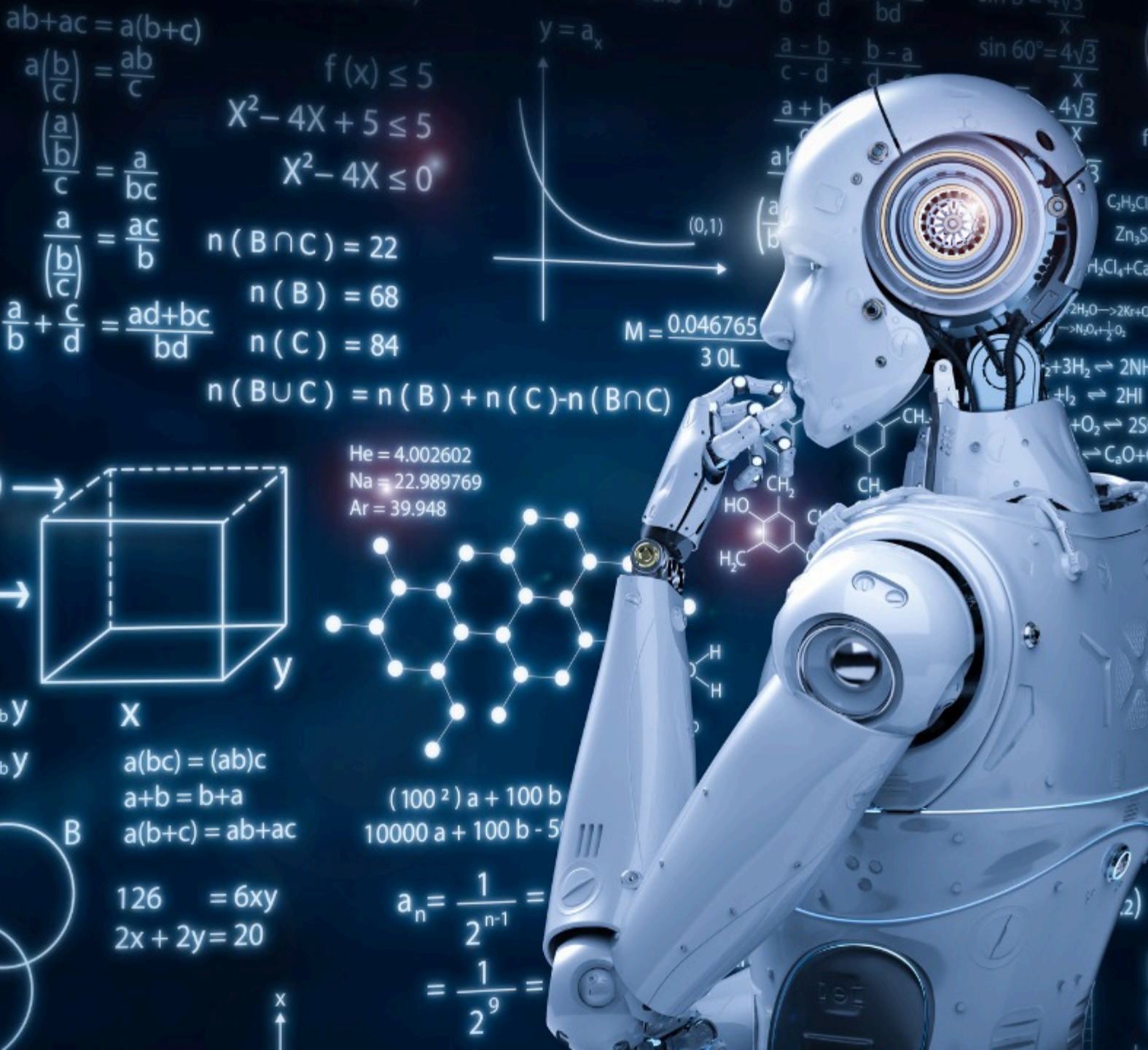




Learning Outcome 3

SORTING ALGORITHMS

PROF. DR. RASHMI GUPTA

RASHMI.GUPTA@KRISTIANIA.NO

Major Objectives

- Bubble Sort
- Insertion Sort
- **Merge Sort**
- **Quick Sort**

Bubble Sort

- ✓ Bubble Sort is the simplest sorting algorithm that works by **repeatedly swapping the adjacent elements if they are in the wrong order.**
- ✓ This algorithm is not suitable for large data sets as its average and worst-case time complexity is quite high.
- ✓ Bubble sort algorithm:
 - ✓ Run a nested for loop to traverse the input array using two variables *i* and *j*, such that $0 \leq i < n-1$ and $0 \leq j < n-i-1$
 - ✓ If **arr[j]** is greater than **arr[j+1]** then swap these adjacent elements, else move on
 - ✓ Print the sorted array

How Does Bubble Sort Work?

Input: $arr[] = \{5, 1, 4, 2, 8\}$

First Pass:

- *Bubble sort starts with the first two elements, comparing them to check which is greater.*
 - $(5\ 1\ 4\ 2\ 8) \rightarrow (1\ 5\ 4\ 2\ 8)$, Here, the algorithm compares the first two elements and swaps since $5 > 1$.
 - $(1\ 5\ 4\ 2\ 8) \rightarrow (1\ 4\ 5\ 2\ 8)$, Swap since $5 > 4$
 - $(1\ 4\ 5\ 2\ 8) \rightarrow (1\ 4\ 2\ 5\ 8)$, Swap since $5 > 2$
 - $(1\ 4\ 2\ 5\ 8) \rightarrow (1\ 4\ 2\ 5\ 8)$, Now, since these elements are already in order ($8 > 5$), algorithm does not swap them.

How Does Bubble Sort Work?

```
BubbleSort(arr):
    n = length(arr)
    for i from 0 to n-1:
        swapped = false # Track if any swaps occur
        for j from 0 to n-1-i:
            if arr[j] > arr[j+1]: # Compare adjacent elements
                Swap(arr[j], arr[j+1]) # Swap if in wrong order
                swapped = true
        if swapped == false:
            Break # Optimization: If no swaps, array is already sorted
```

- The algorithm needs one **whole** pass without any swap to know it is sorted.
 - (1 2 4 5 8) -> (1 2 4 5 8)
 - (1 2 4 5 8) -> (1 2 4 5 8)
 - (1 2 4 5 8) -> (1 2 4 5 8)
 - (1 2 4 5 8) -> (1 2 4 5 8)

Understand the complexity of bubble sort [here](#).

2. Bubble Sort (Step-by-Step)

Algorithm:

- Repeatedly swap adjacent elements if they are in the wrong order.

Sorting Process:

Initial List:

[5, 1, 4, 2, 8]

Pass 1:

- Compare 5 and 1 → Swap → [1, 5, 4, 2, 8]
- Compare 5 and 4 → Swap → [1, 4, 5, 2, 8]
- Compare 5 and 2 → Swap → [1, 4, 2, 5, 8]
- Compare 5 and 8 → No swap

After Pass 1: [1, 4, 2, 5, 8]

Pass 2:

- Compare 1 and 4 → No swap
- Compare 4 and 2 → Swap → [1, 2, 4, 5, 8]
- Compare 4 and 5 → No swap
- Compare 5 and 8 → No swap

After Pass 2: [1, 2, 4, 5, 8]

Pass 3:

- Compare 1 and 2 → No swap
- Compare 2 and 4 → No swap
- Compare 4 and 5 → No swap

After Pass 3: [1, 2, 4, 5, 8] (Sorted)

Final Sorted Array (Bubble Sort):

[1, 2, 4, 5, 8]

Illustration

`arr = {23, 1, 10, 5, 2}`

Initial:

- Current element is **23**
- The first element in the array is assumed to be sorted.
- The sorted part until **0th** index is : **[23]**

First Pass:

- Compare **1** with **23** (current element with the sorted part).
- Since **1** is smaller, insert **1** before **23** .
- The sorted part until **1st** index is: **[1, 23]**

Second Pass:

- Compare **10** with **1** and **23** (current element with the sorted part).
- Since **10** is greater than **1** and smaller than **23** , insert **10** between **1** and **23** .
- The sorted part until **2nd** index is: **[1, 10, 23]**

Third Pass:

- Compare **5** with **1** , **10** , and **23** (current element with the sorted part).
- Since **5** is greater than **1** and smaller than **10** , insert **5** between **1** and **10** .
- The sorted part until **3rd** index is : **[1, 5, 10, 23]**

Fourth Pass:

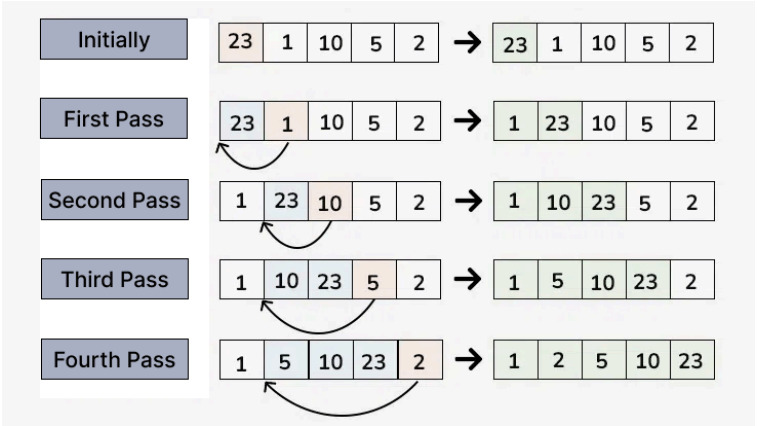
- Compare **2** with **1, 5, 10** , and **23** (current element with the sorted part).
- Since **2** is greater than **1** and smaller than **5** insert **2** between **1** and **5** .
- The sorted part until **4th** index is: **[1, 2, 5, 10, 23]**

Final Array:

- The sorted array is: **[1, 2, 5, 10, 23]**

Algorithm Steps

1. **Start with the second element** (since a single-element list is already sorted).
2. **Pick an element** and compare it with the elements before it.
3. **Shift larger elements one position to the right** to make space for the current element.
4. **Insert the element** into its correct position.
5. **Repeat the process** for all elements in the list.



```
InsertionSort(arr):
    for i from 1 to length(arr) - 1:
        key = arr[i] # Pick the element to insert
        j = i - 1    # Start comparing with previous elements

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j] # Shift elements to the right
            j = j - 1

        arr[j + 1] = key # Insert the element into its correct position
```

1. Insertion Sort (Step-by-Step)

Algorithm:

- Start from the second element (index **1**) and insert it into the correct position in the sorted part of the array.

Sorting Process:

Initial List:

[5, 1, 4, 2, 8]

Pass	Current Key	Comparisons & Shifts	Modified Array
1	1	5 > 1 → Shift 5	[1, 5, 4, 2, 8]
2	4	5 > 4 → Shift 5	[1, 4, 5, 2, 8]
3	2	5 > 2 → Shift 5 4 > 2 → Shift 4	[1, 2, 4, 5, 8]
4	8	No shifts needed	[1, 2, 4, 5, 8]

Final Sorted Array (Insertion Sort):

[1, 2, 4, 5, 8]

Insertion Sort

✓ Insertion sort is a simple sorting algorithm that works by **iteratively inserting each element of an unsorted list into its correct position in a sorted portion of the list.**

✓ **Time Complexity of Insertion Sort**

✓ **Best case:** $O(n)$, If the list is already sorted, where n is the number of elements in the list.

✓ **Average case:** $O(n^2)$, If the list is randomly ordered

✓ **Worst case:** $O(n^2)$, If the list is in reverse order

✓ Java and python Implementation: [Link](#)

Comparison of both Bubble Sort and Insertion Sort Time Complexity

Comparison of When to Use Each Algorithm

Factor	Insertion Sort	Bubble Sort
Best for	Small datasets or nearly sorted data	Teaching sorting & detecting small disorder in lists
Time Complexity	$O(n)$ best case, $O(n^2)$ worst case	$O(n^2)$ in all cases
Efficiency	More efficient for small data	Less efficient but useful for educational purposes
Use Case Example	Auto-sorting emails in an inbox	Checking if a dataset is already sorted

Comparison of Both Algorithms:

Sorting Method	Best Case Complexity	Worst Case Complexity	Stability
Insertion Sort	$O(n)$	$O(n^2)$	Stable
Bubble Sort	$O(n^2)$	$O(n^2)$	Stable

Both **Insertion Sort** and **Bubble Sort** successfully sorted `[5,1,4,2,8]` into `[1,2,4,5,8]` . However, **Insertion Sort** is generally more efficient than **Bubble Sort** for small and nearly sorted datasets.

Pen - and – Paper Exercise

Have a nice shuffle of card deck and get random cards.

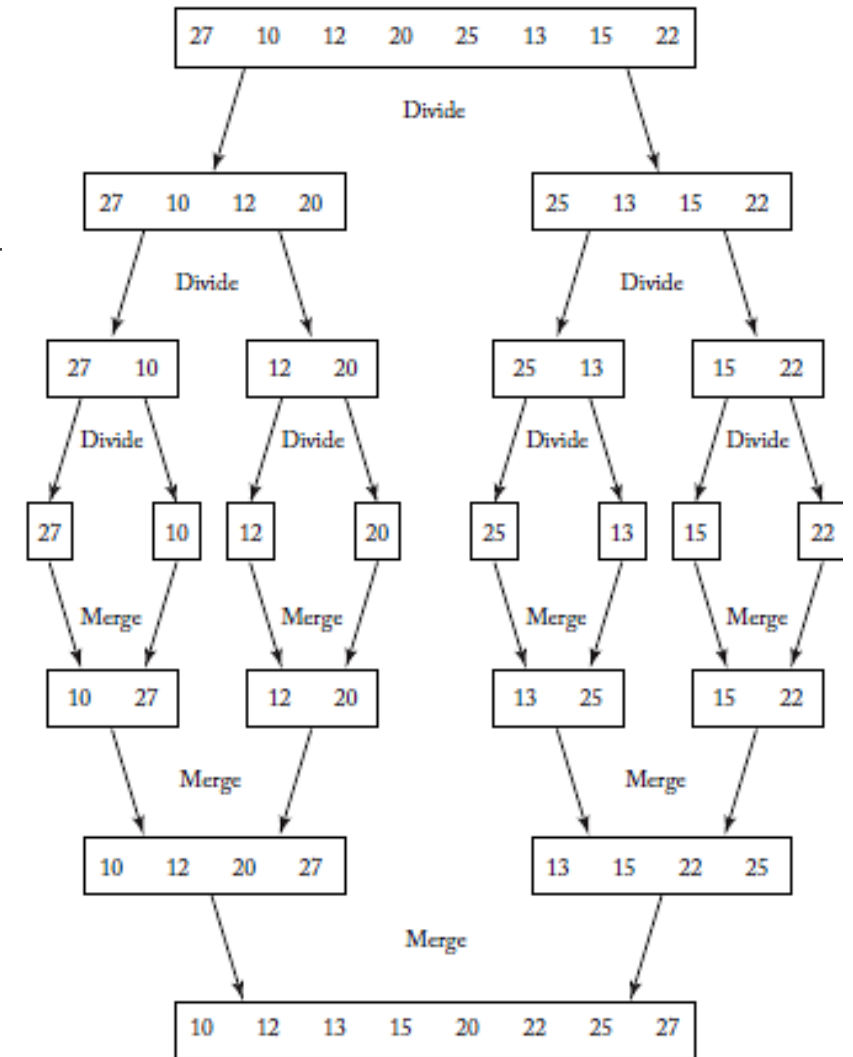
Now, try sorting the card using:

- Bubble Sort
- Insertion Sort

Which algorithm you find more efficient?

Merge Sort

- ✓ Sort an array S of size n (for simplicity, let n be a power of 2)
- ✓ **Divide** an array S into 2 sub-arrays each with $n/2$ items
- ✓ **Conquer (solve)** recursively sort each sub-array until array is sufficiently small (size 1, terminal condition: merging begins)
- ✓ **Combine** merge the solutions to the sub-arrays into a single sorted array



Merge Sort

- ✓ Merges the two arrays U and V created by the recursive calls to mergesort
- ✓ Input size
 - ✓ h the number of items in U
 - ✓ m the number of items in V
- ✓ Basic operation:
 - ✓ Comparison of U[i] to V[j]

• Table 2.1 An example of merging two arrays U and V into one array S*

<i>k</i>	<i>U</i>	<i>V</i>	<i>S</i> (Result)
1	10 12 20 27	13 15 22 25	10
2	10 12 20 27	13 15 22 25	10 12
3	10 12 20 27	13 15 22 25	10 12 13
4	10 12 20 27	13 15 22 25	10 12 13 15
5	10 12 20 27	13 15 22 25	10 12 13 15 20
6	10 12 20 27	13 15 22 25	10 12 13 15 20 22
7	10 12 20 27	13 15 22 25	10 12 13 15 20 22 25
—	10 12 20 27	13 15 22 25	10 12 13 15 20 22 25 27 ← Final values

```

MERGE-SORT(A, n)
1. If n < 2, return // Base case: single element is already sorted
2. mid = n / 2
3. Create leftArray[0..mid-1] and rightArray[0..n-mid-1]
4. Copy A[0..mid-1] into leftArray
5. Copy A[mid..n-1] into rightArray
6. MERGE-SORT(leftArray, mid)
7. MERGE-SORT(rightArray, n - mid)
8. MERGE(A, leftArray, rightArray, mid, n - mid)

MERGE(A, leftArray, rightArray, leftSize, rightSize)
1. i = 0, j = 0, k = 0
2. While i < leftSize and j < rightSize
3.   If leftArray[i] ≤ rightArray[j]
4.     A[k] = leftArray[i]
5.     i = i + 1
6.   Else
7.     A[k] = rightArray[j]
8.     j = j + 1
9.     k = k + 1
10. While i < leftSize
11.   A[k] = leftArray[i]
12.   i = i + 1, k = k + 1
13. While j < rightSize
14.   A[k] = rightArray[j]
15.   j = j + 1, k = k + 1
  
```

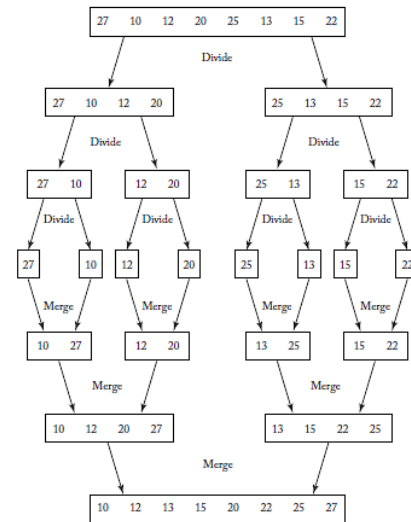
Worst-Case Time Complexity Analysis

- ✓ $W(n)$ = time to sort U + time to sort V + time to merge
- ✓ $W(n) = W(h) + W(m) + h+m-1$
- ✓ **First analysis assumes n is a power of 2**
 - ✓ $h = \lfloor n/2 \rfloor = n/2$
 - ✓ $m = n - h = n - n/2 = n/2$
 - ✓ $h + m = n/2 + n/2 = n$
- ✓ $W(n) = W(n/2) + W(n/2) + n - 1 = 2W(n/2) + n-1$ for $n > 1$ and n a power of 2
- ✓ $W(1) = 0$
- ✓ $W(n) = n \lg n - (n-1) \in \Theta(n \lg n)$
- ✓ **n not a power of 2**
- ✓ $W(n) = W(\lfloor n/2 \rfloor) + W(\lceil n/2 \rceil) + n-1$
- ✓ $W(n) \in \Theta(n \lg n)$

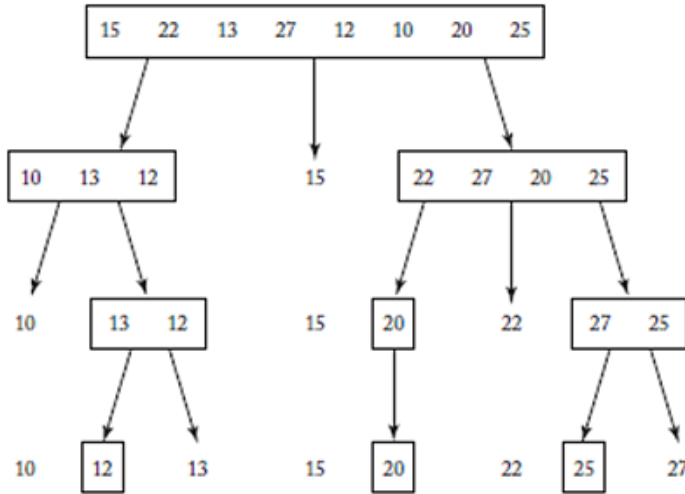
Pen-and-Paper Exercise

Use Mergesort to sort the following list. Show the actions step by step.

123 34 189 56 150 12 9 240



Quicksort (Partition Exchange Sort)



Algorithm 2.6

Quicksort

Problem: Sort n keys in nondecreasing order.

Inputs: positive integer n , array of keys S indexed from 1 to n .

Outputs: the array S containing the keys in nondecreasing order.

```
void quicksort (index low, index high)
{
    index pivotpoint;

    if (high > low){
        partition(low, high, pivotpoint);
        quicksort(low, pivotpoint - 1);
        quicksort(pivotpoint + 1, high);
    }
}
```

- ✓ Array recursively divided into two partitions and recursively sorted
- ✓ Division based on a **pivot item**, it can be any item, for simplicity choose first item as pivot item
- ✓ pivot item divides the array into two sub-arrays
- ✓ All items $<$ pivot placed in sub-array before pivot
- ✓ All items $\geq$ pivot placed in sub-array after pivot
- ✓ Basic Operation:
 - ✓ Comparison of $S[i]$ with pivot item
 - ✓ Input size n

Worst-Case Complexity Analysis of Quicksort

- ✓ Array is sorted in non-decreasing order
- ✓ Array is repeatedly sorted into an empty sub-array
 - ✓ 0 = left sub-array
 - ✓ $n-1$ = right sub-array
 - ✓ pivot item
- ✓ Worst case Time Complexity
 - ✓ $T(n)$ = time to sort left sub-array $T(0)$ + time to sort right sub-array $T(n-1)$ + time to partition $(n-1)$
- ✓ From B16
 - ✓ $T(n) = n(n-1)/2$
 - ✓ Use induction to show it is the worst case
 - ✓ $W(n) \leq n(n-1)/2 = O(n^2)$ (When pivot is always the smallest or largest element)

Pen-and-Paper Exercise

Use Quicksort to sort the following list. Choose different Pivot selection methods:

- ✓ First element as pivot
- ✓ Last element as pivot
- ✓ Random element as pivot

Show the actions step by step

123 34 189 56 150 12 9 240

